Deep Learning for Visual Computing - Assignment 3

Andjelija Vasiljevic - 12450320
Alicia Schartmann - 12331461

June 2026

1. What did you?

This project presents a comparative analysis of three state-of-the-art YOLO object detection models (YOLOv5, YOLOv8, and YOLOv11) applied to brain tumor detection in magnetic resonance imaging (MRI) scans. The primary objective was to implement and evaluate these three model generations for detecting and localizing tumors across four distinct classes: glioma, meningioma, pituitary tumors, and non-tumor cases.

The dataset used in this study is the "MRI for Brain Tumor with Bounding Boxes" `https://www.kaggle.com/datasets/ahmedsorour1/mri-for-brain-tumor-with-bounding-boxes` dataset from Kaggle, comprising 5.249 annotated MRI images across four tumor classes. The dataset was initially divided into training and validation sets, and a dedicated test set was created by randomly sampling 20% of the training data, resulting in a final split of 3.789 training images, 512 validation images, and 948 test images. The class distribution across the dataset is relatively balanced, with the training set containing 1.153 glioma cases, 1.449 meningioma cases, 1.424 pituitary tumor cases, and 711 non-tumor cases, while the validation and test sets maintain a similar proportional distribution. All images are provided in the YOLO format with normalized bounding box coordinates (center x, center y, width, height), enabling direct implementation without additional preprocessing or annotation conversion.

Three medium-variant YOLO models (YOLOv5m, YOLOv8m, and YOLOv11m) were selected to ensure consistent model capacity while evaluating architectural improvements across generations. YOLOv5, developed by Ultralytics, employs a CSPDarknet backbone with Path Aggregation Network (PAN) for multi-scale feature fusion, utilizing anchor-based predictions with a standard convolutional detection head. YOLOv8 introduces significant architectural refinements, including a redesigned backbone with C2f modules that replace C3 blocks, improved feature pyramid networks with decoupled detection heads, and an anchor-free detection paradigm that predicts object center and bounding box dimensions directly. YOLOv11 further enhances the architecture by introducing advanced neural network design techniques, incorporating spatial-channel feature interaction modules, and optimizing the backbone and neck components for improved feature representation and computational efficiency.

All three models were trained using identical hyperparameter configurations to ensure a fair comparison across architectures. The training protocol employed a batch size of 16 images, an input resolution of 640×640 pixels, and the Adam optimizer with a learning rate of 0.001, momentum of 0.937, and weight decay of 0.0005. Training was conducted for a maximum of 100 epochs with an early stopping mechanism using a patience of 20 epochs, monitoring the validation mean Average Precision (mAP) at 0.50 IoU threshold

as the stopping criterion to prevent overfitting. Data augmentation techniques including random horizontal flipping, mosaic augmentation, and color jittering were applied during training to enhance model robustness and generalization.

Model evaluation was conducted using standard object detection metrics computed on the held-out test set containing 948 images unseen during training and validation. The primary metric employed was mean Average Precision at 0.50 IoU threshold (mAP@50), which measures the average precision across all four tumor classes at the standard IoU threshold of 0.50. Additional metrics computed include mAP@50-95 (average across IoU thresholds from 0.50 to 0.95 in increments of 0.05), per-class Average Precision at 0.50 IoU (AP@50 for each tumor class individually), per-class precision (ratio of true positive detections to all positive predictions), and per-class recall (ratio of true positive detections to all ground truth positive instances). Per-class metrics enable assessment of whether models exhibit detection bias toward particular tumor types.

2. Where did you download the code (including links)? Did you change anything and (if so) what? Did you have problems running the code? If so, how did you fix them?

The YOLO models were sourced from the official Ultralytics GitHub repositories, which represent the primary and most actively maintained implementations of these object detection architectures. The unified ultralytics Python package (version 8.0+) provides API access to YOLOv5, YOLOv8, and YOLOv11 models through a consistent interface. The complete source code is publicly available at the following repositories:

- YOLOv5: `https://github.com/ultralytics/yolov5` with model: `https://github.com/ultralytics/yolov5/blob/master/models/yolov5m.yaml`
- YOLOv8: `https://github.com/ultralytics/ultralytics` with `https://github.com/ultralytics/ultralytics/blob/main/ultralytics/cfg/models/v8/yolov8.yaml`
- YOLOv11: `https://github.com/ultralytics/ultralytics` with `https://github.com/ultralytics/ultralytics/blob/main/ultralytics/cfg/models/11/yolo11.yaml`

We utilized the core Ultralytics functionality for model training, validation on test sets, and results.csv metric outputs, but created significant custom modifications including: (1) custom trainer classes (YOLOv5Trainer, YOLOv8Trainer, YOLOv11Trainer) to provide a unified interface across all three model versions, (2) a patched_yaml() context manager to handle absolute path requirements for model training, and (3) custom metrics computation using manual IoU calculations and 11-point AP interpolation via the DetectionMetrics class (metrics.py). Additionally, we developed train.py with a MODEL_REGISTRY to orchestrate sequential training of all three models with identical hyperparameters, test.py with _find_best_weights() to locate and evaluate the most recent trained checkpoints, and comparison.py with custom visualization functions to generate side-by-side prediction comparisons. With the help of these changes, the standard Ultralytics library was changed from a single-model training tool into a comprehensive framework that could compare three YOLO versions on brain tumor detection with detailed per-class metrics.

Problems encountered

The most significant challenge encountered during training was hardware-related. Training deep learning models of this scale is computationally demanding and places considerable strain on the GPU. A CUDA out of memory error occurred during training with the default batch size of 16, which was resolved by reducing the batch size to 8. Additionally, a shared memory error (Windows error code 1455) caused YOLOv5 training to crash at epoch 91 due to the GPU and system memory being under sustained load.

3. Which tests did you perform and what are the result? Discuss the results and include images and figures.

All three models were evaluated on the held-out test set of 947 images across four tumor classes: Glioma, Meningioma, NoTumor, and Pituitary. Evaluation metrics include mean Average Precision at IoU threshold 0.50 (mAP@50), mAP averaged over IoU thresholds 0.50 to 0.95 (mAP@50-95), Precision, and Recall.

Model	mAP@50	mAP@50-95	Precision	Recall
YOLOv5m	0.9457	0.7600	0.9248	0.9208
YOLOv8m	0.9378	0.7487	0.9236	0.9032
YOLOv11m	**0.9505**	**0.7633**	0.9189	**0.9280**

Table 1: Test set evaluation results across all three YOLO model variants.

Class	YOLOv5m	YOLOv8m	YOLOv11m
Glioma	0.8690	0.8650	**0.8885**
Meningioma	**0.9831**	0.9760	0.9823
NoTumor	0.9945	0.9863	**0.9948**
Pituitary	0.9361	0.9238	**0.9364**

Table 2: Per-class AP@50 on the test set.

YOLOv11m achieved the highest overall mAP@50 of 0.9505, outperforming both YOLOv5m (0.9457) and YOLOv8m (0.9378). YOLOv11m achieves this with fewer parameters (20M) compared to YOLOv5m and YOLOv8m (both approximately 25M), showing improved architectural efficiency in the latest generation. YOLOv8m performed slightly below YOLOv5m, likely because early stopping was triggered at epoch 73, suggesting the model may have benefited from additional training epochs.

Across all three models, Glioma was consistently the most challenging class, with AP@50 values ranging from 0.865 to 0.889. This is expected, as gliomas are diffuse and irregular in shape, making precise bounding box localization more difficult compared to other tumor types. NoTumor achieved near-perfect scores across all models (above 0.986), reflecting the visual absence of tumor structures in those scans. Meningioma and Pituitary tumors also showed strong and consistent performance across all three architectures.

Figure 1 shows the validation mAP@50 curves across training epochs for all three models. All three models converge to above 0.9 mAP@50 within the first 30 epochs. YOLOv8m converged fastest in the early epochs, while YOLOv11m showed a notable dip around epochs 10 to 15 before recovering and ultimately achieving the highest final validation mAP@50.

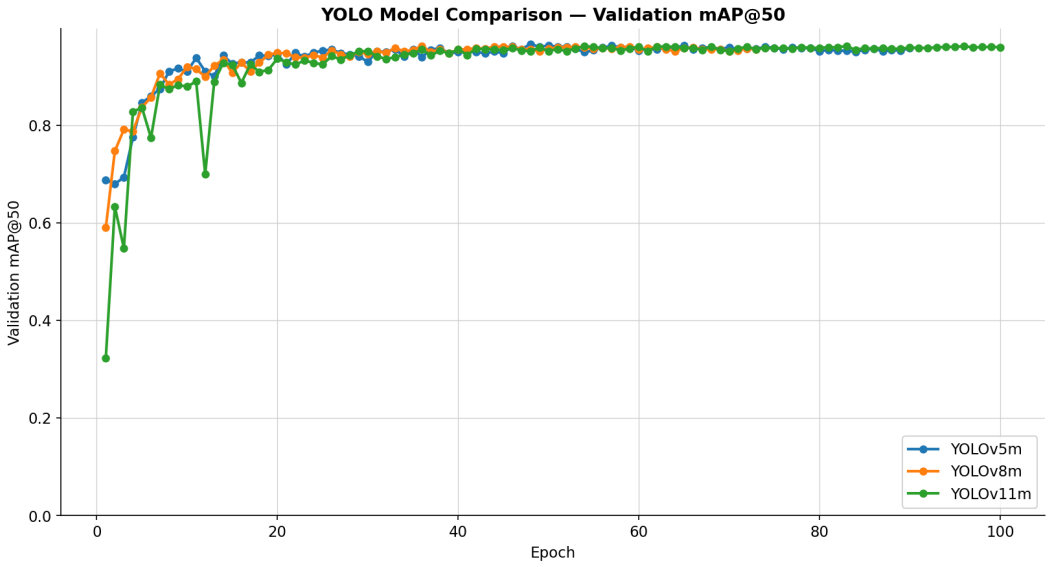


Figure 1: Validation mAP@50 over training epochs for YOLOv5m, YOLOv8m, and YOLOv11m.

Figure 2 provides a more detailed view of training, showing both loss and mAP curves across all three models. All models exhibit similar loss reduction patterns, with improvement in the first 20 epochs followed by gradual refinement. The training loss of YOLOv11m continues to decrease through all 100 epochs, whereas YOLOv5m and YOLOv8m plateau earlier, consistent with early stopping triggering sooner for those models.

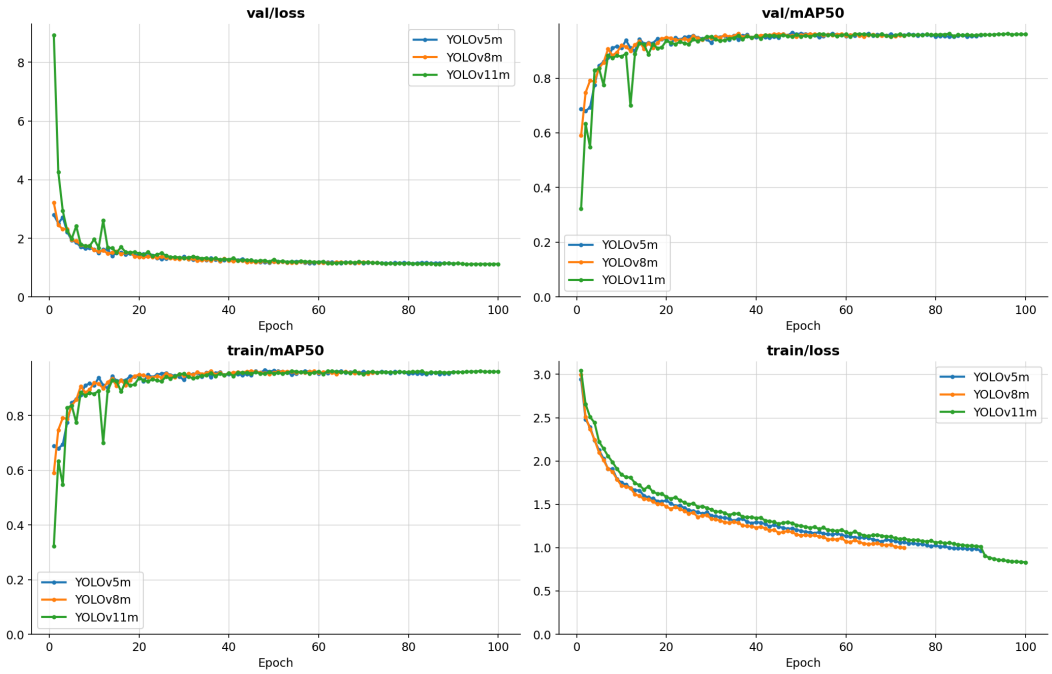


Figure 2: Training and validation loss and mAP@50 curves for all three YOLO model variants.

Figure 3 shows sample predictions from YOLOv11m on the test set. The model accurately localizes tumors across all four classes, with predicted bounding boxes closely matching the ground truth annotations.

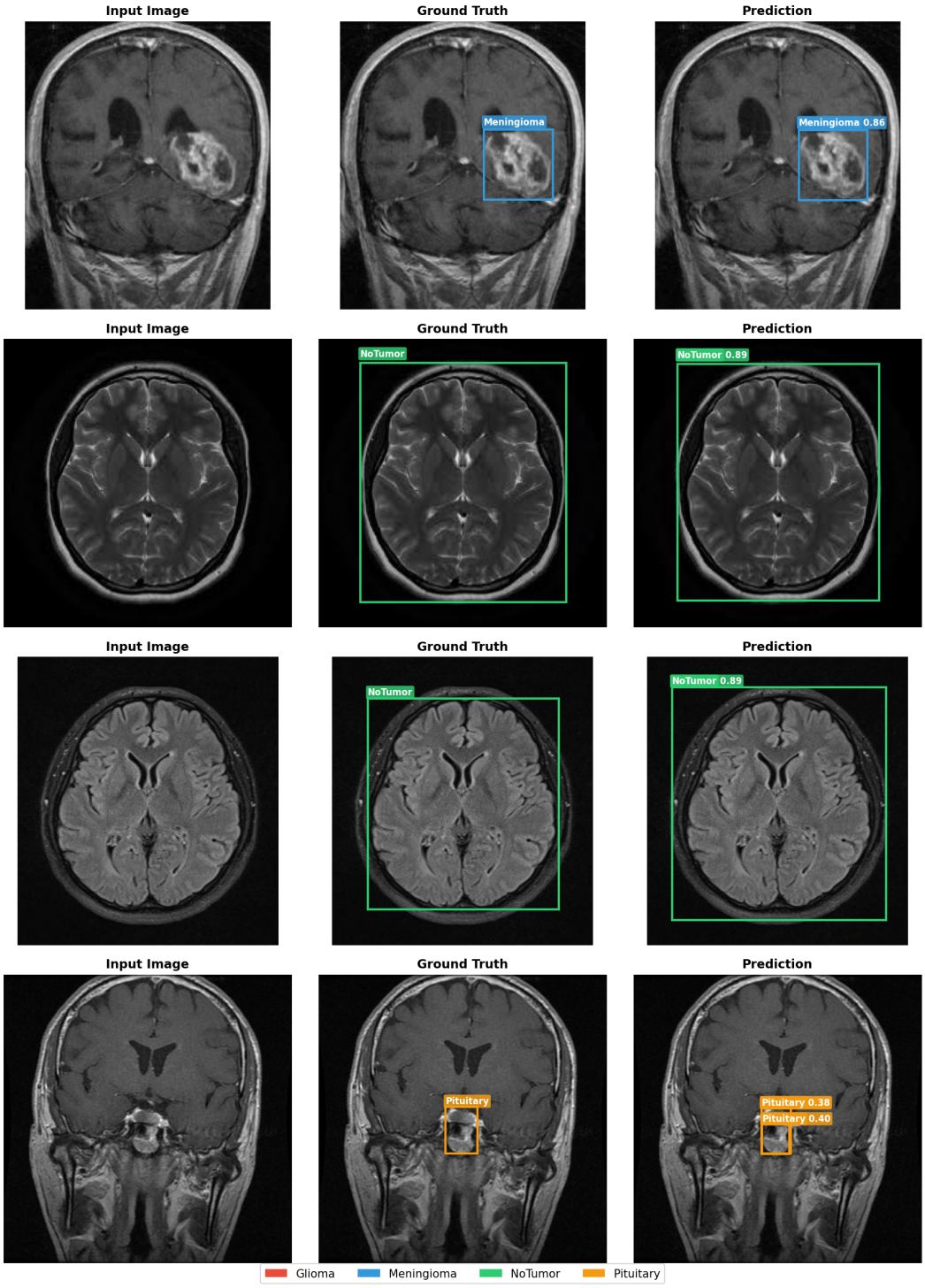


Figure 3: Sample test set predictions by YOLOv11m. Each row shows the input MRI image, ground truth bounding box, and the model's predicted bounding box with confidence score.